

MODULE 10 · DOMAIN 5 - BEYOND SQL

When It Is Not SQL

The same job, written in Spark. Three failure modes that have no SQL equivalent.

DOMAIN 5 of 5 MODULE 10 of 11 PROMPTS 3

What changes, and what does not

STILL TRUE

Carries over

- Grain decides everything
- Counts bisect the problem
- Documents drift from code
- Argued is not proved

NEW HERE

No SQL equivalent

- Types coerce silently on join
- Errors surface far from the cause
- Code depends on state not in the file
- Config and code disagree

THE TIEBREAKER

The engine will not check your join keys. In SQL it at least tries.

Assert the schema. The engine will not do it for you.

Two types, no error

Parts read from Parquet. Movements read from a CSV export.

```
01 parts = spark.read.parquet("/lake/pmats")      # partno: bigint
02 moves = spark.read.csv("/drop/wmov", header=True) # partno: string
03
04 df = parts.join(moves, on="partno", how="left")
05
06 -- Spark casts one side to make the join work.
07 -- No error. No warning. The job succeeds.
08 -- Leading zeros in the CSV never match the bigint.
09 -- Result: consumption is zero for 40% of parts.
```

WHY IT IS WORSE HERE

In SQL this often throws. In Spark it is a successful run with a wrong answer.

What you actually type

Ask for assertions, not for a review of the join.

```
Below is a PySpark job. Before any transformation runs, I want
the schema asserted rather than assumed.
```

```
For every DataFrame created by a read, write an assertion that
fails loudly if the schema is not what the code expects.
```

- Column present, and its exact dtype
- For every join key: same dtype on both sides
- Row count is greater than zero

```
Then list every join where Spark would silently cast one side,
and say which side it would cast.
```

WHY ASSERT RATHER THAN REVIEW

Assertions are permanent. A review finds it once, then the file changes.

The error is not where the stack trace points.

Why the trace misleads

- ◆ **Transformations do not run.** They build a plan. Nothing executes yet.
- ◆ **The action runs everything.** A count or a write, hundreds of lines later.
- ◆ **The trace names the action.** Not the transformation that was wrong.
- ◆ **So paste the whole job.** A model given only the failing line cannot find the cause.

THE POINT

This is why 'here is my error, here is the line' works in SQL and fails here.

What you actually type

Give it the whole lineage. Ask it to walk backwards.

This PySpark job fails at the write on line 214. The full job is below. Do not assume the fault is at line 214.

Walk backwards through the lineage from the action. List every transformation contributing to it, in reverse order.

For each, state:

- What it assumes about its input
- Whether that assumption is checked anywhere
- What the error would look like if it were the cause

Then rank them by how well each explains the actual error.

THE BULLET THAT NARROWS IT

Most candidates would produce a different error. That eliminates them fast.

The Spark version of count triage

- ◆ **Insert an action between stages.** A count forces execution up to that point.
- ◆ **Count after every read, and after every join.** Same bisect as module five.
- ◆ **It costs a full recompute each time.** Cache first, or sample the input.
- ◆ **Then the counts read exactly like SQL.** Up means fan-out, down means a filter.

IN PRACTICE

Same technique, one extra step. You have to force execution before you can count anything.

The notebook is not the program.

It runs for you. Only for you.

A notebook that has been open for three weeks.

```
01 # Cell 4
02 df = df.filter(F.col("site").isin(active_sites))
03
04 # active_sites was defined in a cell that has been deleted.
05 # It still exists in the kernel. The notebook still runs.
06 # On a fresh kernel it fails. In the scheduler it fails.
07
08 # Cell 9
09 spark.conf.set("spark.sql.shuffle.partitions", "8")
10 # Set after the job was tuned. Affects every cell above it
11 # on the next run, and none of them on this one.
```

THE GAP

A model reading this file sees a program. You are running something else entirely.

What you actually type

Ask what a fresh kernel would do.

Below is a notebook, exported in cell order. Assume it runs top to bottom on a fresh kernel, by a scheduler, with no prior state.

List everything referenced before it is defined, in that order.
Include variables, imports, temp views and config settings.

Then list anything defined after it is first used, and which earlier cells would behave differently on a rerun.

Then: which cells depend on tables this notebook does not create.

WHY IT WORKS

Cell order is a fact in the file. Execution order is not.

Consumption comes back as zero for about 40% of parts. The job succeeded. No errors in the log.

The same logic in SQL, against the same source data, returns correct figures.

What do you check first?

- ☐ A The filter logic, since a filter is removing those parts
- ☐ B The dtypes on both sides of every join key
- ☐ C Whether the CSV source is missing those rows
- ☐ D Row counts after each stage

◆ PAUSE AND DECIDE

What do you check first?

A The filter logic

A filter would remove rows. These parts are present with a zero, which is a failed match.

C Missing source rows

The SQL version reads the same source and finds them. So they are there.

B Join key dtypes

SQL works and Spark does not, on identical data. That points at the layer SQL types for you.

D Stage counts

Useful, and a left join keeps the rows. Counts would look correct throughout.

THE SIGNAL

Zero rather than absent is the tell. The row survived the join and matched nothing.

A job fails at a write on line 214 with a null pointer error. You paste lines 205 to 214 and the error message.

You get three suggestions. None of them fixes it.

What went wrong with your approach?

- ☐ A The error message was too generic to act on
- ☐ B You pasted the action, not the transformations that built it
- ☐ C You should have pasted the full stack trace
- ☐ D The model cannot debug Spark reliably

◆ PAUSE AND DECIDE

What went wrong with your approach?

A Generic error

It is generic, and a generic error with full lineage is still diagnosable.

C Full stack trace

The trace describes Spark internals below your action. It still will not name your transformation.

B Pasted the action only

Ten lines around the write contain none of the transformations that produced the bad column.

D Cannot debug Spark

Same conclusion people reach about SQL review. It is the input that was wrong, not the tool.

THE SIGNAL

In SQL, the failing line is usually the wrong line. In Spark, it is almost never the wrong line's fault.

A notebook has run daily for two months. You schedule it and it fails immediately on a name error.

It still runs perfectly when you execute it yourself.

What is the most likely cause?

- ☐ A The scheduler has a different Python environment
- ☐ B Something is defined in your kernel and not in the file
- ☐ C Cell execution order differs under the scheduler
- ☐ D A permissions difference on the service account

◆ PAUSE AND DECIDE

What is the most likely cause?

- A

 Different environment
That usually raises an import error, not a name error on your own variable.
- B

 Kernel-only definition
A name error plus works-for-me is the signature of state that is not in the file.
- C

 Different cell order
Schedulers run top to bottom. It is your own order that has been non-linear for two months.
- D

 Permissions
Permissions fail with access errors. They do not produce a name error.

THE SIGNAL

Two months of daily runs is how long the kernel has been alive, not how long the code has worked.

You add schema assertions to a job. On the first scheduled run, an assertion fails: a column that was a string is now a decimal.

The job would have run fine. Spark would have handled the cast.

Was the assertion worth having?

- ☐ A No. It failed a job that would have worked
- ☐ B Yes. The upstream schema changed and nobody told you
- ☐ C Only if the cast would have changed a result
- ☐ D Yes, but the assertion should warn rather than fail

◆ PAUSE AND DECIDE

Was the assertion worth having?

- A

 Not worth it
Judges the assertion by this one cast. Its job is telling you the contract moved.
- B

 Worth it, schema changed
An upstream change reached you without notice. Next time it may not be a harmless cast.
- C

 Only if results changed
You cannot know that in advance, which is exactly why the assertion runs first.
- D

 Warn, do not fail
A warning in a scheduled log is a warning nobody reads. This is the monitor argument again.

THE SIGNAL

Same shape as the zero threshold from module eight. Detection stays exact; routing is a separate question.

Take this to work tomorrow

- ◆ Add dtype assertions on both sides of every join key. Today, in one job.
- ◆ When something fails at an action, paste the whole lineage. Never the failing line.
- ◆ Run every notebook once on a fresh kernel before you schedule it.
- ◆ Counts still bisect. You just have to force execution first.

THE PAYOFF

The first one takes ten minutes and removes the most common silent failure in Spark.

The job is correct. Now something has to
run it.